Docent: Dirk Hostens

Apps for Android



Opdracht: Navigation in Jetpack Compose

Bronnen:

Zie Toledo

In een reeks van opdrachten maken we een app om een lijst van todo items te beheren. We zullen taken aanmaken en toekennen aan gebruikers en uiteraard ook een gebruikersbeheer voorzien. Elke opdracht zal verder bouwen op de app van de vorige opdracht.

Na deze opdracht ken je de Navigation component van JetPack Compose.

In deze opdracht

- Leer je om een correcte architectuur van je app op te zetten
- Maak je gebruik van de Navigation component

In deze opdracht zullen we de navigatie toevoegen tussen de verschillende schermen. De app start met een lijst van ToDo items. Als ik op een item klik, kom ik op het detailscherm. Daar kunnen we navigeren naar AddEditToDo waarbij we het geselecteerde ToDO item kunnen aanpassen. Met de back knop keren we terug naar het vorig scherm. Op het scherm met de lijst van ToDo items kan je ook navigeren naar AddEditToDo scherm om een nieuw ToDo item toe te voegen.

De eerste stap is het toevoegen van de correcte libraries. Daarna kan je aan Gemini vragen wat de volgende stappen zijn om de correcte naviagtie op te zetten. Doe dit stapsgewijs en test/compileer je project na elke stap.

Indien je Compile errors hebt, kan je via rechterklik op de error vragen aan Gemini hoe dit eventueel kan opgelost worden.

Veel succes!

Wees extra kritisch op het eindresultaat. In mijn geval werd ToDoId nog steeds als parameter doorgegeven aan de composable ToDoDetailScreen die het op zijn beurt via een Factory class doorgaf aan het ViewModel. Met de introductie van de Navigation library wordt ToDo nu als navigatie argument doorgegeven en kan dit rechtstreeks opgehaald worden in het ViewModel.

Dit was de composable function ToDoDetailScreen en het viewmodel voor de wijziging.

```
@Composable
fun ToDoDetailScreen(
      toDoId: Int,
      modifier: Modifier = Modifier,
      viewModel: ToDoDetailViewModel = viewModel(factory =
            ToDoDetailViewModelFactory(toDoId)),
      onNavigateBack: () -> Unit,
      onNavigateToEdit: (Int) -> Unit
      ) {
      …
}

class ToDoDetailViewModel(private val toDoId: Int) : ViewModel() { … }
```

Na de wijziging:

```
@Composable
fun ToDoDetailScreen(
      modifier: Modifier = Modifier,
      viewModel: ToDoDetailViewModel = viewModel(factory =
            ToDoDetailViewModelFactory),
      onNavigateBack: () -> Unit,
      onNavigateToEdit: (Int) -> Unit
      ) {
      …
}

class ToDoDetailViewModel(private val savedStateHandle: SavedStateHandle
) : ViewModel() {

      private val toDoId: Int = checkNotNull(savedStateHandle["toDoId"])
      ….

}
```

Hetzelfde geldt voor AddEditToDoView en Viewmodel. Ook hier mag ToDoId niet meer worden meegegeven.